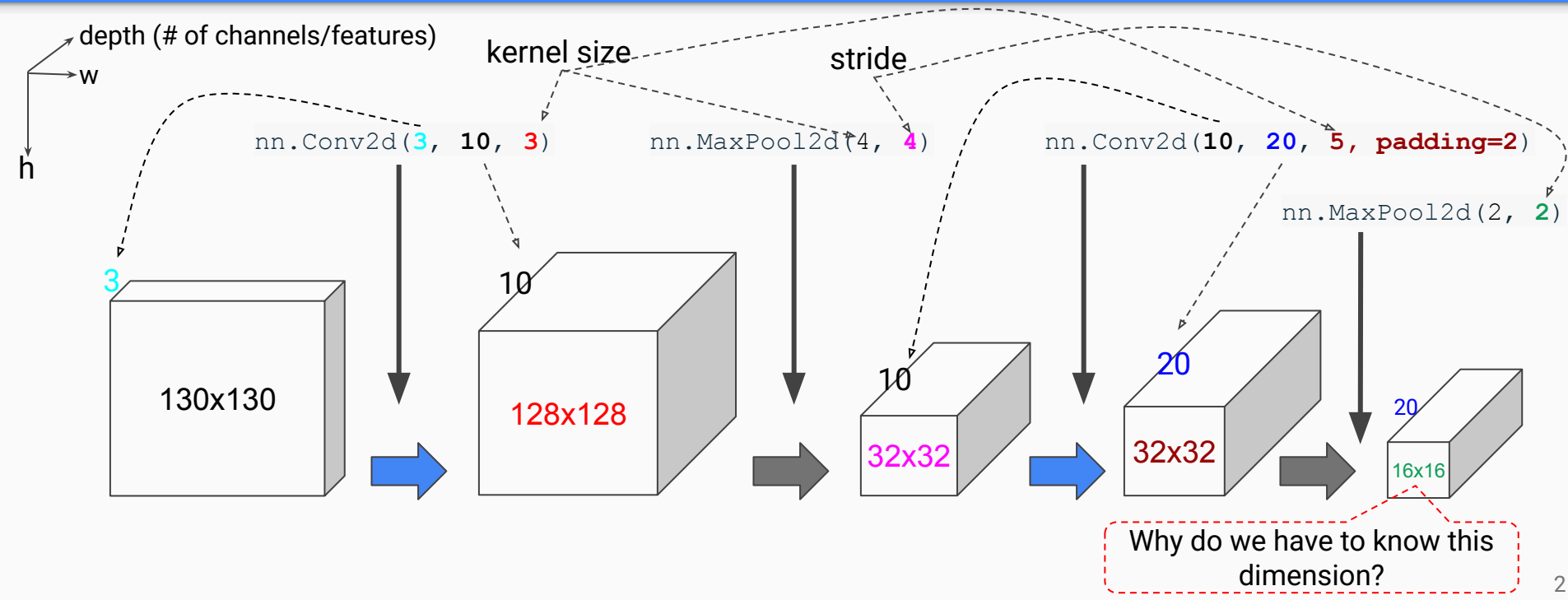


CNN Milestones

CSCI 611 Applied Machine Learning, Week 4 Session 2
Bo Shen

Visualize Conv/MaxPooling Layers

- How many trainable weights (blue arrow)?



Conv Kernel is **Space Invariant**, but **Channel/Feature Variant**

How many weights to train in the CNN?

- `nn.Conv2d(3, 10, 3)`

$$\Rightarrow 3 \times (3 \times 3) \times 10 = \text{inCh} \times (\text{kw} \times \text{kh}) \times \text{outCh}$$

- `nn.MaxPool2d(4, 4)`

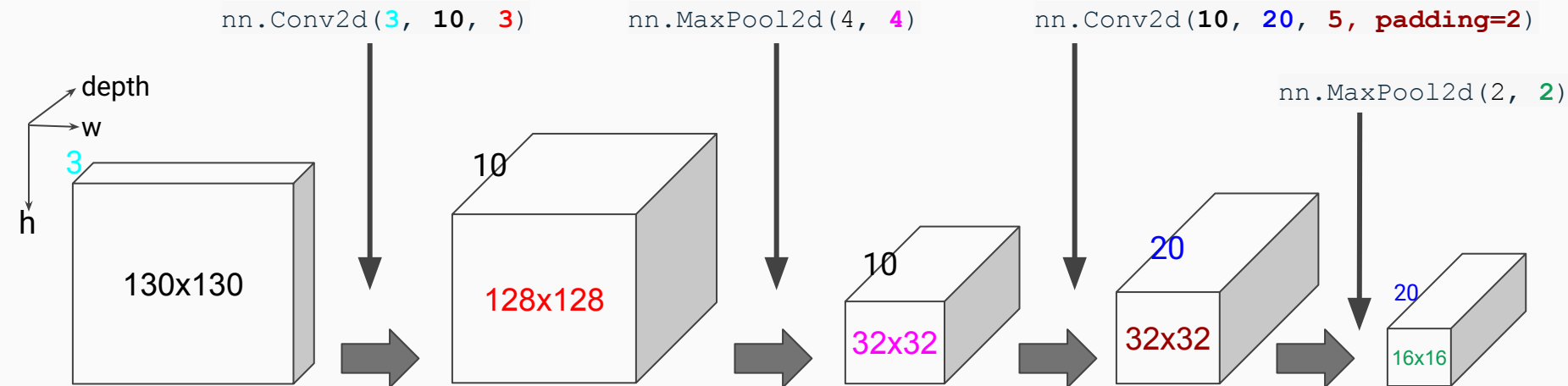
$$\Rightarrow 0$$

- `nn.Conv2d(10, 20, 5, padding=2)`

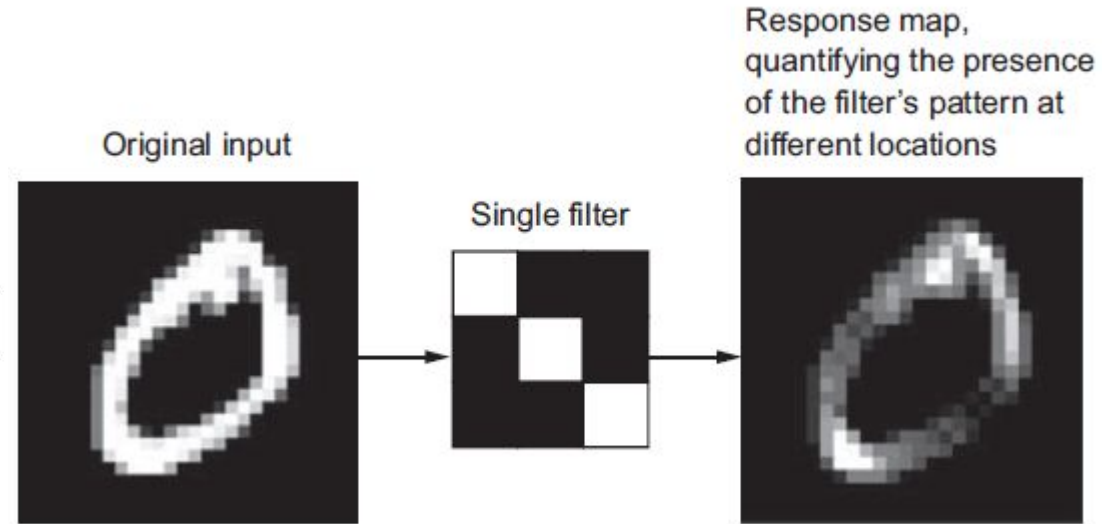
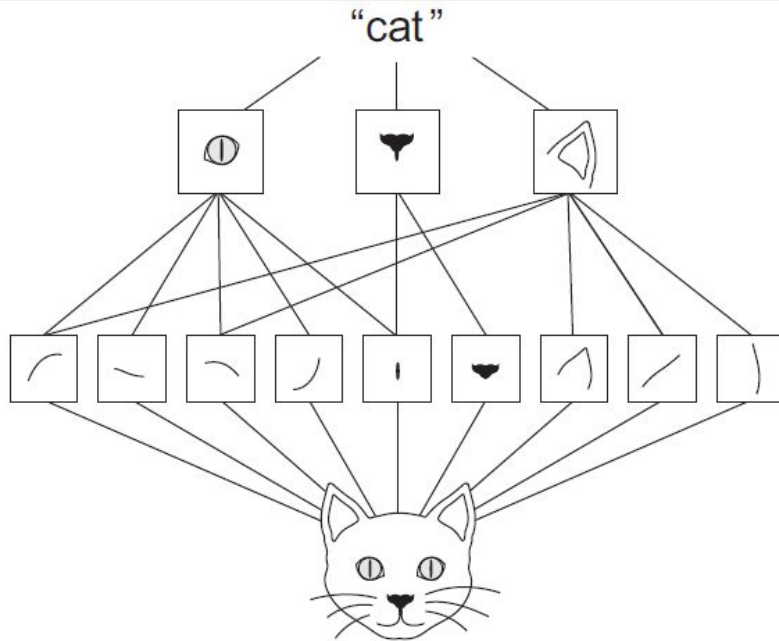
$$\Rightarrow 10 \times (5 \times 5) \times 20$$

- `nn.MaxPool2d(2, 2)`

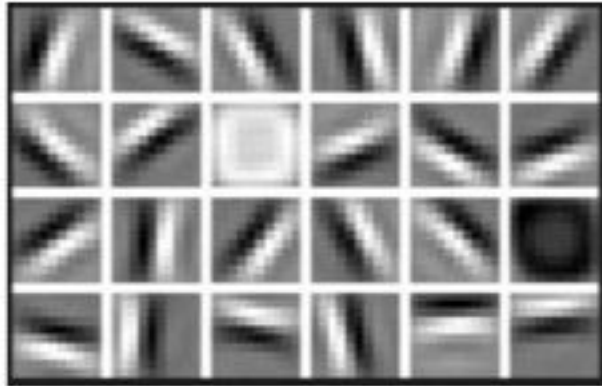
$$\Rightarrow 0$$



CNN vs. Convolution in Traditional CV



Multiscale Feature Extraction



First Layer Representation



Second Layer Representation



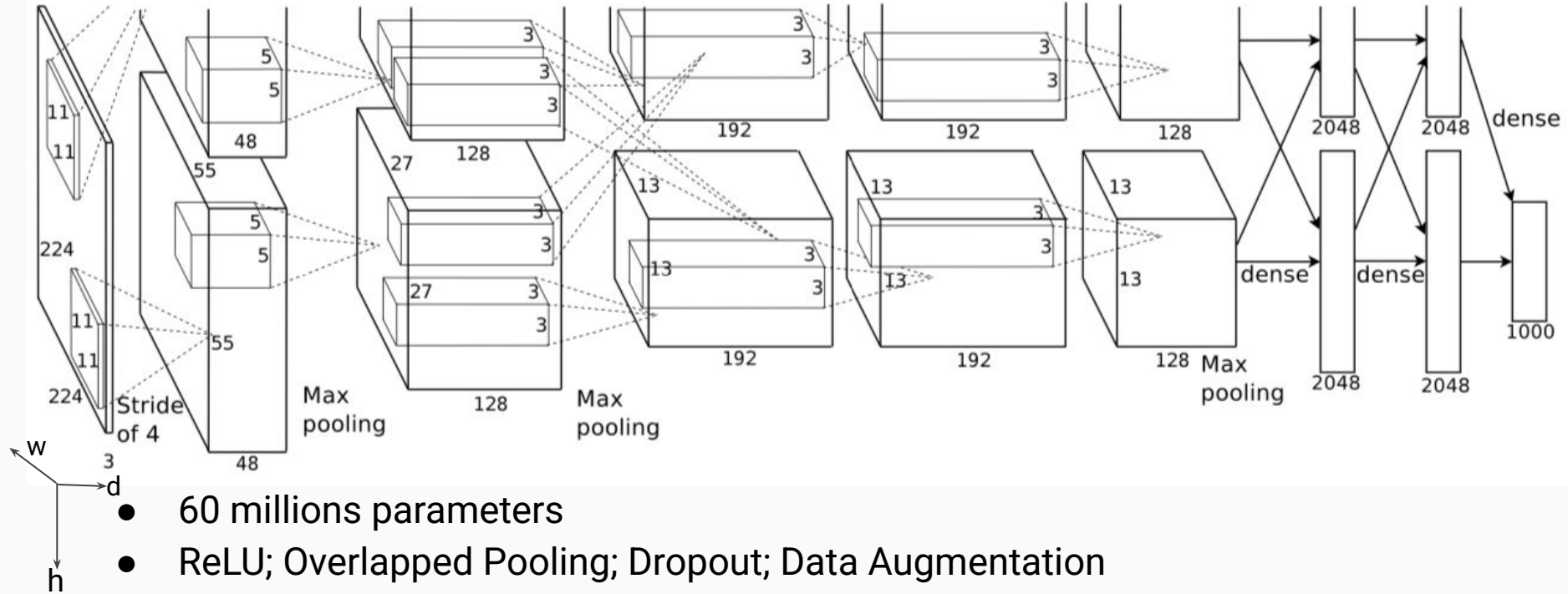
Third Layer Representation

- AlexNet (2012, U. of Toronto, 11 layers):
 - ReLU, dropout, 11x11 conv window
 - <http://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf>
 - (ImageNet) Top-5 error rate: 15.3%
- VGG (2014, VGG16, VGG19, Oxford U, 16-19 layers):
 - 3x3 exclusively with 2x2 maxpooling
 - <https://arxiv.org/pdf/1409.1556.pdf>
 - (ImageNet) Top-5 error rate: 8.47%
- ResNet (2015, Microsoft Research, 159 layers):
 - Vanishing gradient → skip layers so that gradient travels in short cuts
 - <https://arxiv.org/pdf/1512.03385v1.pdf>
 - (ImageNet) Top-5 error rate: 4.49%



error rate halved twice

AlexNet [<http://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf>]



- 60 millions parameters
- ReLU; Overlapped Pooling; Dropout; Data Augmentation
- Grouped convolution separates the top and bottom parts, thus the training can happen in parallel on two separate GPUs (2012, GPU is not powerful enough, and memory is limited – 3GB, not enough for AlexNet)

AlexNet (detailed visual on architecture)

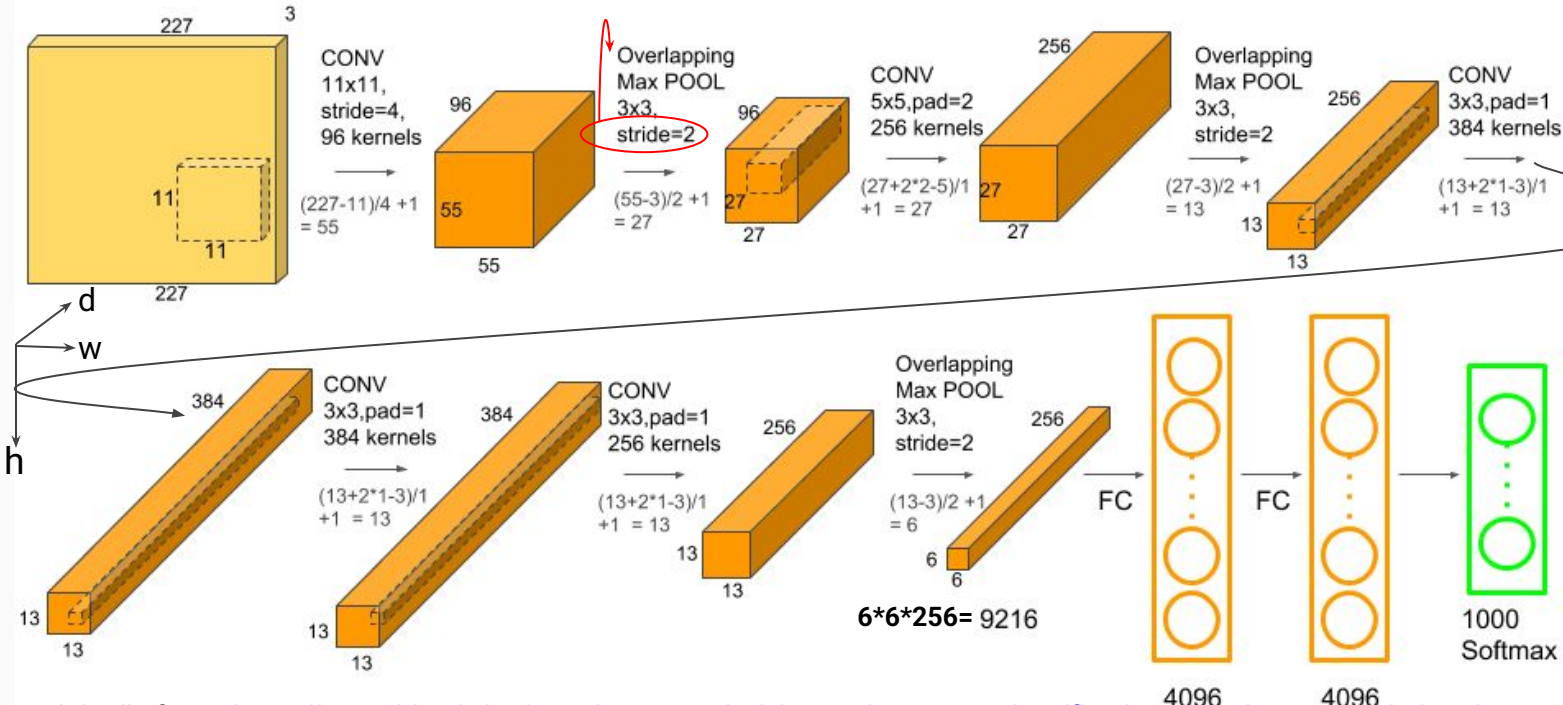
<https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

<https://pytorch.org/docs/stable/generated/torch.nn.MaxPool2d.html>

$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times \text{padding}[0] - \text{dilation}[1] \times (\text{kernel_size}[1] - 1) - 1}{\text{stride}[1]} + 1 \right\rfloor$$

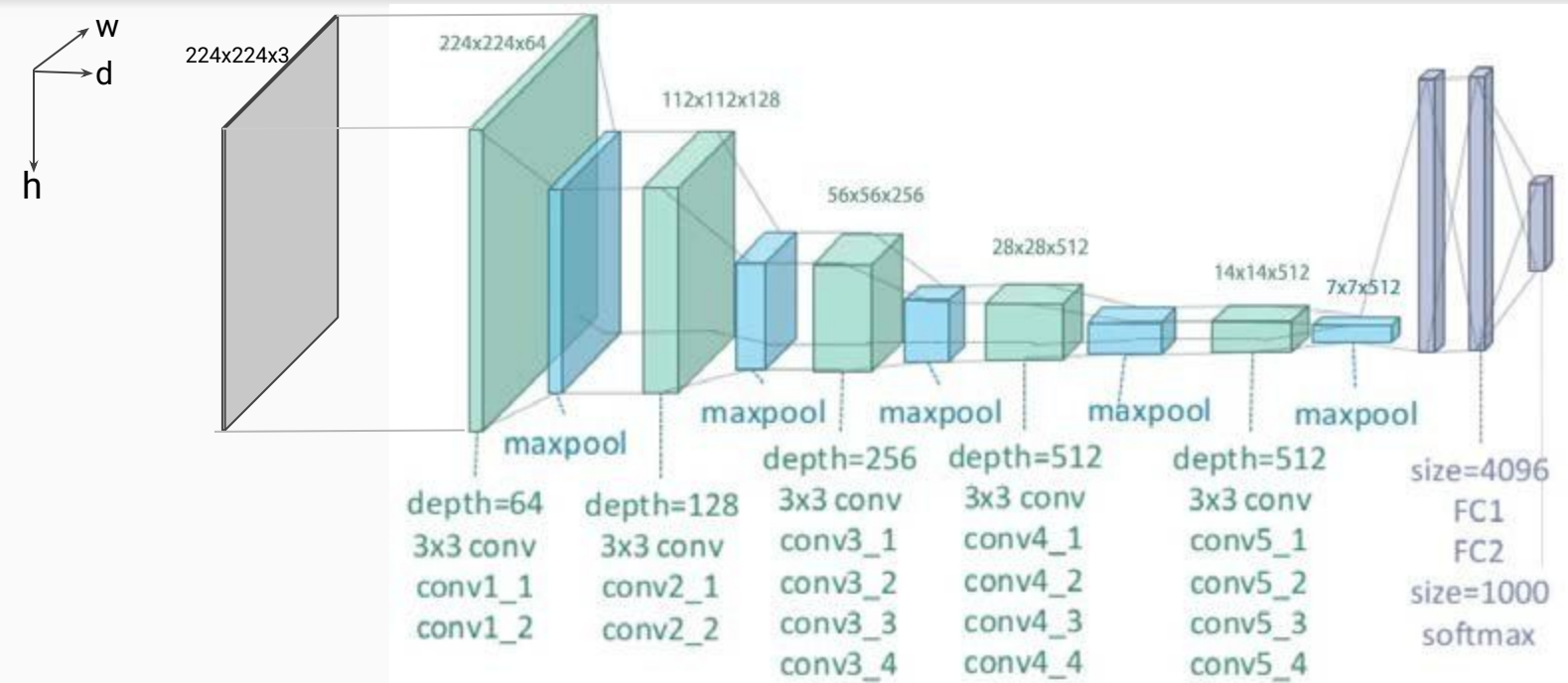
$$W_{out} = \left\lfloor \frac{W_{in} + 2 * \text{padding}[0] - \text{dilation}[1] \times (\text{kernel_size}[1] - 1) - 1}{\text{stride}[1]} + 1 \right\rfloor$$

Default:
padding=0
dilation=1



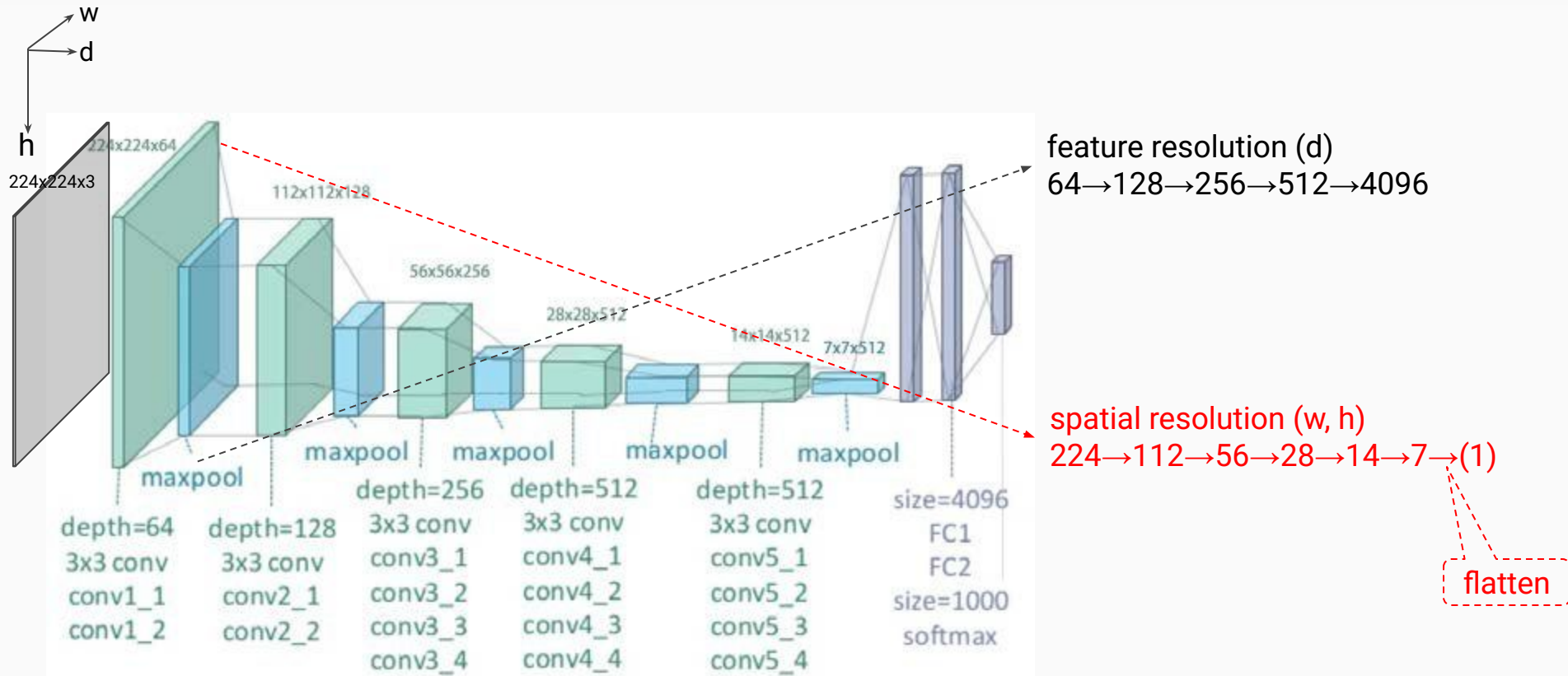
originally from: <https://neurohive.io/en/popular-networks/alexnet-imagenet-classification-with-deep-convolutional-neural-networks/>

VGG19 [<https://arxiv.org/pdf/1409.1556.pdf>]



- 16 conv layers (conv1_1 to conv5_4) + 3 fully connected layers = 19 layers
- all small kernel (3x3)

VGG19 - Typical CNN Design Methodology



VGG19 - Complexity

of out
Channel

of
weights
to train

kw*kh is
always 3x3

Activation = ReLU
on accumulation
of WX

size (wxh)	# of inChannel	Layer name	#Filters	#Parameters	#Activations	kw*kh*inChannel*outChannel	ow*oh*outChannel
224x224	3	input			150K		224*224*3=150K
224x224	3	conv1_1	64	1.7K	3.2M	3*3*3*64=1.7K	224*224*64=3.2M
224x224	64	conv1_2	64	36K	3.2M	3*3*64*64=36.8K	224*224*64=3.2M
112x112	64	max pooling			802K		112*112*64=802K
112x112	64	conv2_1	128	73K	1.6M		
112x112	128	conv2_2	128	147K	1.6M		
56x56	128	max pooling			401K		
56x56	128	conv3_1	256	300K	802K		
56x56	256	conv3_2	256	600K	802K		
56x56	256	conv3_3	256	600K	802K		
56x56	256	conv3_4	256	600K	802K		
28x28	256	max pooling			200K		
28x28	256	conv4_1	512	1.1M	401K		
28x28	512	conv4_2	512	2.3M	401K		
28x28	512	conv4_3	512	2.3M	401K		
28x28	512	conv4_4	512	2.3M	401K		
14x14	512	max pooling			100K		
14x14	512	conv5_1	512	2.3M	100K		
14x14	512	conv5_2	512	2.3M	100K		
14x14	512	conv5_3	512	2.3M	100K		
14x14	512	conv5_4	512	2.3M	100K		
7x7	512	max pooling			25K		
4096	1	fc6		103M	4K		
4096	1	fc7		17M	4K		
1000	1	output		4M	1K		

VGG19 - Complexity

of out
Channel

of
weights
to train

kw*kh is
always 3x3

Activation = ReLU
on accumulation
of WX

size (wxh)	# of inChannel	Layer name	#Filters	#Parameters	#Activations	kw*kh*inChannel*outChannel	ow*oh*outChannel
224x224	3	input			150K		224*224*3=150K
224x224	3	conv1_1	64	1.7K	3.2M	3*3*3*64=1.7K	224*224*64=3.2M
224x224	64	conv1_2	64	36K	3.2M	3*3*64*64=36.8K	224*224*64=3.2M
112x112	64	max pooling			802K		112*112*64=802K
112x112	64	conv2_1	128	73K	1.6M	3*3*64*128=73K	112*112*128=1.6M
112x112	128	conv2_2	128	147K	1.6M	3*3*128*128=147K	112*112*128=1.6M
56x56	128	max pooling			401K		
56x56	128	conv3_1	256	300K	802K		
56x56	256	conv3_2	256	600K	802K	...	...
56x56	256	conv3_3	256	600K	802K		
56x56	256	conv3_4	256	600K	802K		
28x28	256	max pooling			200K		
28x28	256	conv4_1	512	1.1M	401K		
28x28	512	conv4_2	512	2.3M	401K		
28x28	512	conv4_3	512	2.3M	401K		
28x28	512	conv4_4	512	2.3M	401K		
14x14	512	max pooling			100K		
14x14	512	conv5_1	512	2.3M	100K		
14x14	512	conv5_2	512	2.3M	100K		
14x14	512	conv5_3	512	2.3M	100K		
14x14	512	conv5_4	512	2.3M	100K		
7x7	512	max pooling			25K		
4096	1	fc6		103M	4K	?	7*7*512=25088
4096	1	fc7		17M	4K		
1000	1	output		4M	1K		

last conv layer → flatten
→ fully connected to next

VGG19 - Complexity

of out
Channel

of
weights
to train

kw*kh is
always 3x3

Activation = ReLU
on accumulation
of WX

size (wxh)	# of inChannel	Layer name	#Filters	#Parameters	#Activations	kw*kh*inChannel*outChannel	ow*oh*outChannel
224x224	3	input			150K		224*224*3=150K
224x224	3	conv1_1	64	1.7K	3.2M	3*3*3*64=1.7K	224*224*64=3.2M
224x224	64	conv1_2	64	36K	3.2M	3*3*64*64=36.8K	224*224*64=3.2M
112x112	64	max pooling			802K		112*112*64=802K
112x112	64	conv2_1	128	73K	1.6M	3*3*64*128=73K	112*112*128=1.6M
112x112	128	conv2_2	128	147K	1.6M	3*3*128*128=147K	112*112*128=1.6M
56x56	128	max pooling			401K		
56x56	128	conv3_1	256	300K	802K		
56x56	256	conv3_2	256	600K	802K	...	...
56x56	256	conv3_3	256	600K	802K		
56x56	256	conv3_4	256	600K	802K		
28x28	256	max pooling			200K		
28x28	256	conv4_1	512	1.1M	401K		
28x28	512	conv4_2	512	2.3M	401K		
28x28	512	conv4_3	512	2.3M	401K		
28x28	512	conv4_4	512	2.3M	401K		
14x14	512	max pooling			100K		
14x14	512	conv5_1	512	2.3M	100K		
14x14	512	conv5_2	512	2.3M	100K		
14x14	512	conv5_3	512	2.3M	100K		
14x14	512	conv5_4	512	2.3M	100K		
7x7	512	max pooling			25K		
4096	1	fc6		103M	4K	7*7*512*4096=102760448	7*7*512=25088
4096	1	fc7		17M	4K		4096
1000	1	output		4M	1K		

last conv layer → flatten
→ fully connected to next

VGG19 - Complexity

of out
Channel

of
weights
to train

kw*kh is
always 3x3

Activation = ReLU
on accumulation
of WX

size (wxh)	# of inChannel	Layer name	#Filters	#Parameters	#Activations	kw*kh*inChannel*outChannel	ow*oh*outChannel
224x224	3	input			150K		224*224*3=150K
224x224	3	conv1_1	64	1.7K	3.2M	3*3*3*64=1.7K	224*224*64=3.2M
224x224	64	conv1_2	64	36K	3.2M	3*3*64*64=36.8K	224*224*64=3.2M
112x112	64	max pooling			802K		112*112*64=802K
112x112	64	conv2_1	128	73K	1.6M	3*3*64*128=73K	112*112*128=1.6M
112x112	128	conv2_2	128	147K	1.6M	3*3*128*128=147K	112*112*128=1.6M
56x56	128	max pooling			401K		
56x56	128	conv3_1	256	300K	802K		
56x56	256	conv3_2	256	600K	802K	...	...
56x56	256	conv3_3	256	600K	802K		
56x56	256	conv3_4	256	600K	802K		
28x28	256	max pooling			200K		
28x28	256	conv4_1	512	1.1M	401K		
28x28	512	conv4_2	512	2.3M	401K		
28x28	512	conv4_3	512	2.3M	401K		
28x28	512	conv4_4	512	2.3M	401K		
14x14	512	max pooling			100K		
14x14	512	conv5_1	512	2.3M	100K		
14x14	512	conv5_2	512	2.3M	100K		
14x14	512	conv5_3	512	2.3M	100K		
14x14	512	conv5_4	512	2.3M	100K		
7x7	512	max pooling			25K		
4096	1	fc6		103M	4K	7*7*512*4096=102760448	7*7*512=25088
4096	1	fc7		17M	4K	?	?
1000	1	output		4M	1K		

last conv layer → flatten
→ fully connected to next

fully connected:
inNode*outNode

VGG19 - Complexity

of out
Channel weights
to train

kw*kh is
always 3x3

Activation = ReLU
on accumulation
of WX

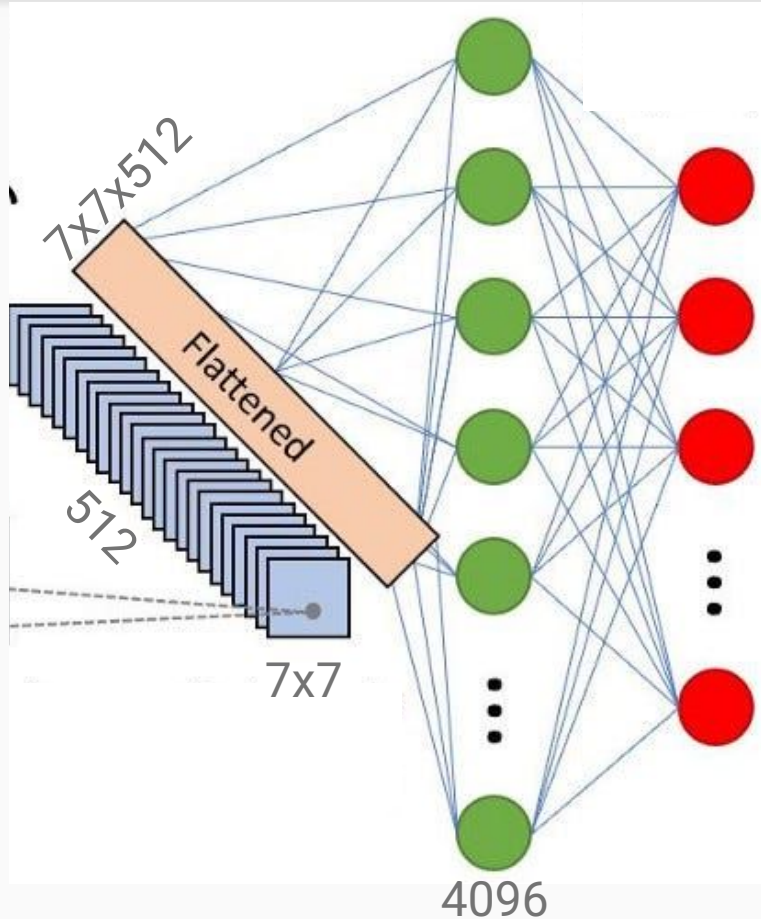
size (wxh)	# of inChannel	Layer name	#Filters	#Parameters	#Activations	kw*kh*inChannel*outChannel	ow*oh*outChannel
224x224	3	input			150K		224*224*3=150K
224x224	3	conv1_1	64	1.7K	3.2M	3*3*3*64=1.7K	224*224*64=3.2M
224x224	64	conv1_2	64	36K	3.2M	3*3*64*64=36.8K	224*224*64=3.2M
112x112	64	max pooling			802K		112*112*64=802K
112x112	64	conv2_1	128	73K	1.6M	3*3*64*128=73K	112*112*128=1.6M
112x112	128	conv2_2	128	147K	1.6M	3*3*128*128=147K	112*112*128=1.6M
56x56	128	max pooling			401K		
56x56	128	conv3_1	256	300K	802K		
56x56	256	conv3_2	256	600K	802K	...	...
56x56	256	conv3_3	256	600K	802K		
56x56	256	conv3_4	256	600K	802K		
28x28	256	max pooling			200K		
28x28	256	conv4_1	512	1.1M	401K		
28x28	512	conv4_2	512	2.3M	401K		
28x28	512	conv4_3	512	2.3M	401K		
28x28	512	conv4_4	512	2.3M	401K		
14x14	512	max pooling			100K		
14x14	512	conv5_1	512	2.3M	100K		
14x14	512	conv5_2	512	2.3M	100K		
14x14	512	conv5_3	512	2.3M	100K		
14x14	512	conv5_4	512	2.3M	100K		
7x7	512	max pooling			25K		
4096	1	fc6		103M	4K	7*7*512*4096=102760448	7*7*512=25088
4096	1	fc7		17M	4K	4096*4096=16777216	4096
1000	1	output		4M	1K	4096*1000=4M	1000

Total > 143M

last conv layer → flatten
→ fully connected to next

fully connected:
inNode*outNode

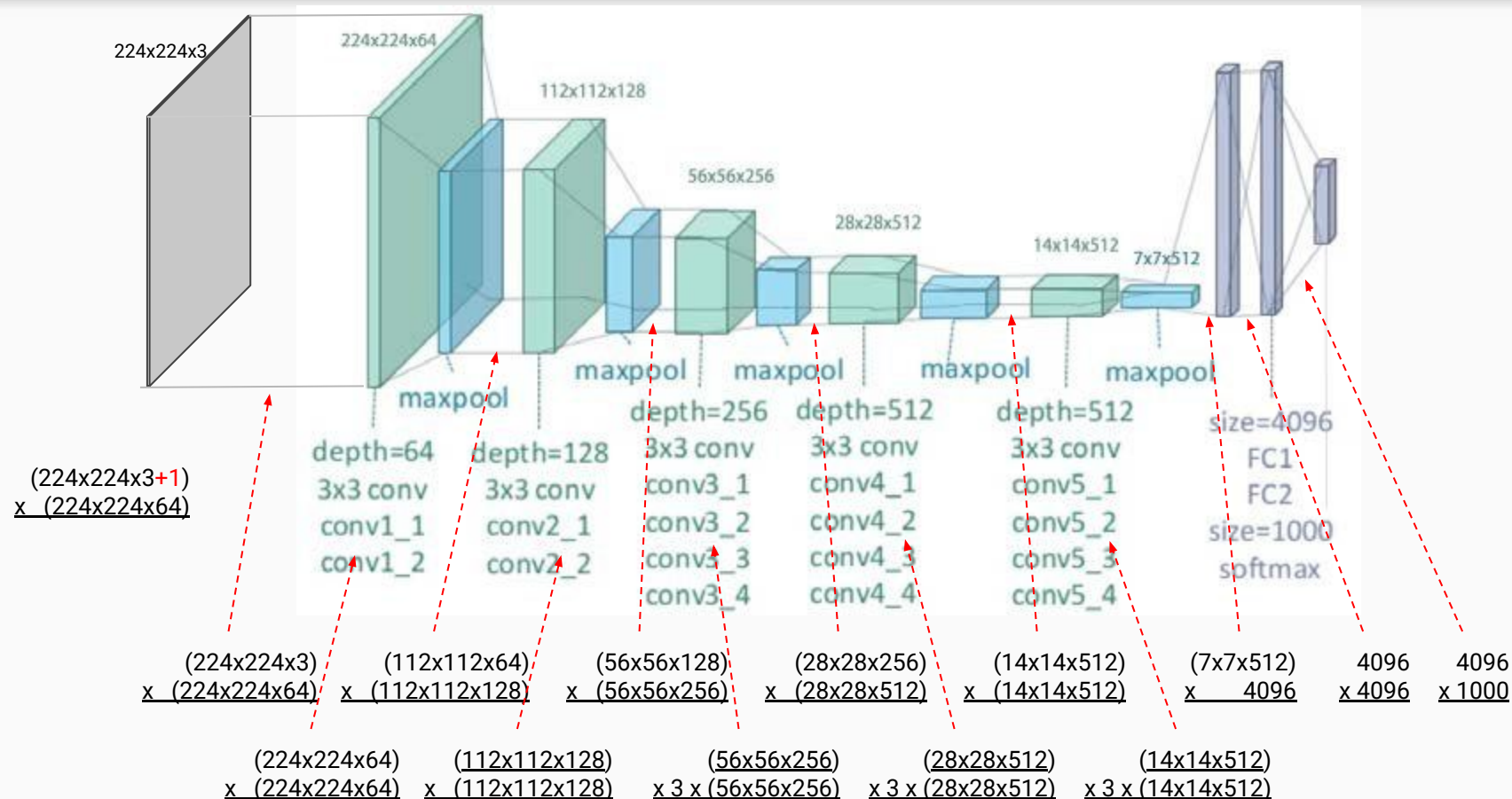
Flattening → Fully Connected



```
model = nn.Sequential(  
    # ...  
    # 28x28x256 → 28x28x512  
    nn.Conv2d(256, 512, 3, 1, 1),  
    nn.ReLU(inplace=True),  
  
    # 28x28 → 14x14  
    nn.MaxPool2d(2, 2),  
  
    # 14x14x512 → 14x14x512  
    nn.Conv2d(512, 512, 3, 1, 1),  
    nn.ReLU(inplace=True),  
  
    # 14x14 → 7x7  
    nn.MaxPool2d(2, 2),  
  
    # Flatten  
    nn.Flatten(),  
  
    # Fully connected  
    nn.Linear(512 * 7 * 7, 4096),  
    nn.ReLU(inplace=True)  
)
```

Need this to correctly
construct the fully
connected layer

What IF Not Using CNN, but All Fully Connected for a VGG19-Like NN? (# of weights)



Q&A